DOCUMENTO TÉCNICO

Sistema de Control de Posición para Panel Solar

Microcontrolador PIC18F45K22 — XC8

Materia: Electrónica Integrada

Microcontrolador: PIC18F45K22

Compilador: MPLAB XC8

Simulación: PROTEUS

Fecha: 08 de Mayo 2026

Luis Alexis Calvillo Deveze

1. Descripción del Sistema y su Funcionamiento

El sistema implementa el control automático de la orientación de un panel solar mediante un servomotor, buscando maximizar la captación de energía solar. Está diseñado para operar en modo bare-metal sobre el microcontrolador PIC18F45K22, sin sistema operativo ni capas de abstracción de hardware externas a la HAL del XC8.

1.1 Modos de Operación

- **Modo AUTO (tecla A):** El sistema lee continuamente el sensor LDR (fotoresistencia) mediante el ADC interno. El ángulo del servomotor se calcula de forma proporcional al nivel de luz: a mayor intensidad luminosa en una dirección, mayor ángulo de rotación hacia ella.
- **Modo MANUAL (tecla B):** El usuario puede posicionar el panel manualmente usando las teclas '2' (incrementar ángulo +10°) y '8' (decrementar ángulo -10°). Se activa con la tecla B.
- **Protección contra viento:** Si la frecuencia del sensor de viento supera el umbral configurado, el sistema coloca el panel en posición horizontal (90°) de forma automática, independientemente del modo activo.

1.2 Flujo General del Programa

El programa sigue el siguiente flujo en su bucle principal:

1. Lectura del teclado matricial 4×4 para detectar cambio de modo o ajuste manual.
2. Lectura del ADC (sensor LDR en RA0) para obtener el nivel de luz ambiental.
3. Evaluación de la variable periodo_viento (actualizada por ISR) para detectar viento peligroso.
4. Cálculo del ángulo del servomotor según el modo activo y el estado del viento.
5. Actualización del ciclo de trabajo del PWM (CCP1) para mover el servomotor.
6. Actualización del LCD con el ángulo actual, lectura de luz y estado del sistema.
7. Retardo de 100 ms y repetición del ciclo.

2. Configuración de Periféricos del PIC18F45K22

2.1 Oscilador Interno (FOSC = 500 kHz)

El microcontrolador utiliza su oscilador interno configurado a 500 kHz, suficiente para las operaciones de control en tiempo real requeridas por el sistema.

```
OSCCON = 0x20;   // IRCF = 001 → 500 kHz
#pragma config FOSC = INTIO67
```

2.2 Módulo ADC — Sensor LDR (RA0 / AN0)

El conversor analógico-digital lee la señal del sensor LDR conectado en RA0. La resolución es de 10 bits (0–1023), justificada a la derecha en los registros ADRESH:ADRESL.

Registro / Bit	Valor y Descripción
ANSELA = 0x01	Solo RA0 configurado como analógico
TRISA = 0x01	RA0 como entrada
ADCON2 = 0xA9	Just. derecha, 12 TAD de adquisición, FOSC/8
ADCON1 = 0x00	Referencias VDD y VSS
ADCON0 = 0x01	Canal AN0 seleccionado, ADC encendido

```
uint16_t leer_luz(void) {
    ADCON0bits.GO = 1;
    while (ADCON0bits.GO);
    return ((uint16_t)ADRESH << 8) | ADRESL;  // Cast explícito para
evitar overflow
}
```

2.3 Módulo CCP1 — PWM para Servomotor (RC2)

El servomotor se controla mediante la señal PWM generada por CCP1. La frecuencia objetivo es ~50 Hz, estándar para servomotores RC.

Cálculo de PR2: Con FOSC = 500 kHz, prescaler TMR2 = 16: PR2 = (FOSC / (4 × prescaler × FPWM)) − 1 = (500000 / (4 × 16 × 50)) − 1 ≈ 155

```
PR2     = 155;      // Período PWM ≈ 50 Hz
T2CON   = 0x06;     // Timer2 ON, prescaler 16
CCP1CON = 0x0C;     // Modo PWM
TRISCbits.TRISC2 = 0;
```

Mapeo de ángulo a duty cycle: 0° → ~1 ms (duty = 31) | 180° → ~2 ms (duty = 62)

```
long duty = (angulo * (62 - 31) / 180) + 31;
CCPR1L = (uint8_t)(duty >> 2);
CCP1CONbits.DC1B = (uint8_t)(duty & 0x03);
```

2.4 Módulo CCP2 — Captura de Frecuencia del Viento (RC1)

El sensor de viento (simulado con generador de señal cuadrada) se conecta a RC1. El módulo CCP2 en modo captura registra el tiempo entre flancos de subida consecutivos, calculando el período de la señal. Un período corto implica alta frecuencia = viento peligroso.

```
TRISCbits.TRISC1 = 1;   // RC1 como entrada
T1CON   = 0x01;          // Timer1 ON, FOSC/4, prescaler 1
CCP2CON = 0x05;          // CCP2: captura en flanco de subida
PIE2bits.CCP2IE  = 1;    // Interrupción CCP2 habilitada
INTCONbits.PEIE  = 1;
INTCONbits.GIE   = 1;
```

La ISR calcula el período restando la captura actual a la anterior:

```
void __interrupt() ISR(void) {
    if (PIR2bits.CCP2IF) {
        static uint16_t ultima_captura = 0;
        uint16_t captura_actual = ((uint16_t)CCPR2H << 8) | CCPR2L;
        periodo_viento = captura_actual - ultima_captura;
        ultima_captura = captura_actual;
        PIR2bits.CCP2IF = 0;
    }
}
```

2.5 Teclado Matricial 4×4 (Puerto B)

El teclado matricial se conecta al Puerto B: RB0–RB3 son salidas (filas) y RB4–RB7 son entradas (columnas) con pull-ups internos habilitados.

```
ANSELB = 0x00;            // Todo digital
TRISB  = 0xF0;            // RB0-RB3: salidas | RB4-RB7: entradas
INTCON2bits.RBPU = 0;     // Pull-ups internos habilitados
LATB   = 0x00;            // Filas en LOW (estado idle)
```

La rutina de lectura activa una fila a la vez (mask de 0 rotando) y verifica qué columna fue jalada a LOW por la tecla presionada. El código resultante combina la nibble de fila (bits 3:0) con la nibble de columna (bits 7:4).

2.6 LCD 2×16 (Puerto D)

La pantalla LCD LM016L se controla a través del Puerto D en modo de 4 bits. Los pines de datos se conectan a RD4–RD7, y los de control RS/E a RD2/RD3.

```
ANSELD = 0x00;   // Puerto D digital
TRISD  = 0x00;   // Todo salidas
// Macros de traducción para lcd.h:
#define RD2 LATDbits.LATD2   // RS
#define RD3 LATDbits.LATD3   // E
#define RD4 LATDbits.LATD4   // D4
// ... RD5, RD6, RD7
```

3. Evidencias de Simulación en PROTEUS

A continuación se describen los estados del sistema observados durante la simulación en PROTEUS:

3.1 Estado: Modo AUTO

El sistema arranca en modo AUTO. El LCD muestra 'Modo: AUTO' en la primera línea y el ángulo calculado junto con la lectura del LDR en la segunda. El servomotor se posiciona según la ecuación: ángulo = (luz × 180) / 1023.

LCD línea 1: Modo: AUTO

LCD línea 2: Ang:090 L:0512

3.2 Estado: Modo MANUAL

Al presionar la tecla B se activa el modo manual. Con la tecla '2' el ángulo aumenta 10° y con '8' disminuye 10°. El rango se limita a [0°, 180°].

LCD línea 1: Modo: MANUAL

3.3 Estado: Alerta de Viento

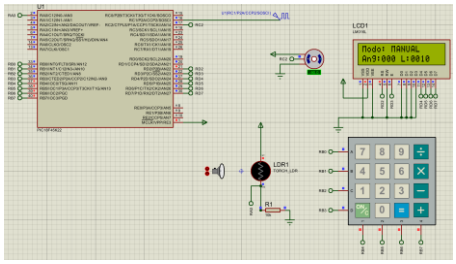
Cuando el generador de señal cuadrada produce una frecuencia alta (período < umbral = 1500 ciclos de Timer1), la variable periodo_viento cae por debajo del umbral y el sistema activa la alerta. El panel se mueve automáticamente a 90° (posición horizontal de seguridad) y el LCD muestra el mensaje de alerta.

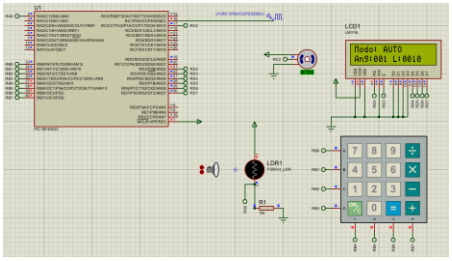
LCD línea 1: ALERTA VIENTO!

Ángulo del servo: 90° (horizontal)

3.4 Arranque del Sistema

Al encender el sistema, el LCD muestra el mensaje 'Iniciando sist..' durante 500 ms, luego limpia la pantalla e inicia el bucle principal.





4. Conclusiones

- **Integración de periféricos heterogéneos:** El proyecto demuestra cómo un único microcontrolador de gama media (PIC18F45K22) puede controlar simultáneamente un ADC, dos módulos CCP, un teclado matricial y una pantalla LCD, aprovechando interrupciones para operaciones críticas en tiempo real.
- **Importancia de las interrupciones:** El uso de la ISR para la captura CCP2 garantiza que ningún pulso del sensor de viento sea ignorado, independientemente de qué tan ocupado esté el bucle principal con otras tareas.
- **Programación bare-metal y sus ventajas:** Al trabajar directamente sobre los registros del hardware sin capas de abstracción, se obtiene control total sobre el temporizado y el consumo de recursos, esencial en sistemas embebidos con recursos limitados.
- **Depuración de errores sutiles:** Errores como el desbordamiento por falta de cast en el ADC o la asignación manual de periodo_viento son difíciles de detectar sin análisis detallado del código. Revisiones sistemáticas de los registros del datasheet son indispensables.
- **Posibles mejoras:** 1) Algoritmo de búsqueda de máximo local (hill-climbing) para el modo AUTO en lugar de mapeo lineal. 2) Histéresis en la detección de viento para evitar falsos positivos. 3) Almacenamiento del ángulo preferido del usuario en EEPROM interna. 4) Comunicación UART para monitoreo remoto del sistema.

5. Referencias

- Microchip Technology Inc. (2010). PIC18F45K22 Data Sheet. DS40001412.
- Microchip Technology Inc. MPLAB XC8 C Compiler User's Guide.
- Labcenter Electronics. PROTEUS Professional — VSM Simulation Documentation.
- Servo Motor Control via PWM — Application Note AN0828, Microchip Technology.